Rapport de Projet

Intelligence Artificielle Industrielle

Système Intelligent de Maintenance Prédictive

Prédiction des défaillances machines par Machine Learning

Ce rapport présente la problématique industrielle, la solution proposée, le pipeline de données, l'entraînement, l'évaluation, l'API, le dashboard, le ML Lab, le déploiement Railway et les résultats opérationnels du système.

Réalisé par : Mohamed Aziz Mansour
Rimel Zouari
Mohamed Ncib

Domaine : Machine Learning, Deep Learning, IoT industriel

Modèle actif : GradientBoosting

Seuil API : 0.25

Année universitaire 2025–2026

Résumé

Ce projet réalise un système complet de maintenance prédictive pour anticiper les pannes de machines industrielles. Le travail part de fichiers CSV bruts, effectue la découverte des données, fusionne les sources, nettoie les incohérences, construit des variables explicables, entraîne plusieurs modèles, sélectionne un modèle actif, expose les prédictions via FastAPI et affiche le résultat dans un frontend synchronisé.

La difficulté principale vient du déséquilibre des classes : après nettoyage, seulement 3.48% des observations correspondent à une panne. Une accuracy brute peut donc être trompeuse. Le projet utilise des métriques adaptées : balanced accuracy, précision, recall, F1, ROC AUC, matrice de confusion et taux d'alerte. Le modèle actif est GradientBoosting avec un seuil de production de 0.25. Il atteint 89.05% de balanced accuracy, 85.94% de précision, 78.57% de pannes capturées et 3.20% de taux d'alerte.

Table des matières

1 Problématique et Solution **1**
1.1 Contexte industriel 1
1.2 Problème de classes déséquilibrées 1
1.3 Solution proposée 2

2 Données, ETL et Nettoyage **3**
2.1 Sources brutes 3
2.2 Fusion canonique 3
2.3 Résultats du nettoyage 3
2.4 Séparation train, validation et test 4

3 Feature Engineering **5**
3.1 Objectif 5
3.2 Variables finales 5
3.3 Analyse de corrélation 6

4 Modélisation et Sélection **7**
4.1 Modèles comparés 7
4.2 Leaderboard 7
4.3 Choix du modèle actif 7

5 Évaluation Finale **8**
5.1 Métriques principales 8
5.2 Matrice de confusion 8
5.3 Courbe ROC et importance des variables 10

6 API, Frontend et Synchronisation **12**
6.1 API FastAPI 12
6.2 Logique de prédiction 12
6.3 Frontend 12
6.4 Synchronisation 12

7 Déploiement Railway **13**
7.1 Préparation 13
7.2 Contenu déployé 13
7.3 Commande CLI 13

8 Conclusion **14**

Table des figures

1.1 Distribution des classes après nettoyage. 2

3.1 Matrice de corrélation des variables. 6

5.1 Matrice de confusion générée. 9
5.2 Courbe ROC du modèle actif. 10
5.3 Importance des variables. 10

Liste des tableaux

2.1 Répartition des pannes après nettoyage. 3
2.2 Séparation stratifiée des données. 4

3.1 Variables construites principales. 5

4.1 Comparaison des modèles candidats. 7

5.1 Métriques finales du modèle actif. 8
5.2 Matrice de confusion finale. 8

Chapitre 1

Problématique et Solution

1.1 Contexte industriel

Les machines industrielles fonctionnent sous des contraintes variables : température, couple, vitesse, usure d'outil et charge mécanique. Une panne imprévue peut arrêter une ligne de production, augmenter les coûts, réduire la qualité et créer un risque de sécurité. La maintenance corrective attend la panne, tandis que la maintenance préventive intervient selon un calendrier fixe. Ces deux approches ne tiennent pas suffisamment compte de l'état réel de la machine.

La maintenance prédictive répond à ce besoin en utilisant les données pour anticiper les défaillances. Le but n'est pas seulement de produire un score de modèle, mais de donner une décision exploitable : continuer la surveillance, planifier une inspection ou inspecter immédiatement.

1.2 Problème de classes déséquilibrées

Le dataset final contient 10 000 lignes, dont seulement 348 pannes. Cela signifie que 96.52% des cas sont normaux. Un modèle qui prédit toujours ń pas de panne ż semble très bon en accuracy, mais il est inutile. Le projet évite ce piège avec la balanced accuracy, la précision, le recall et la matrice de confusion.

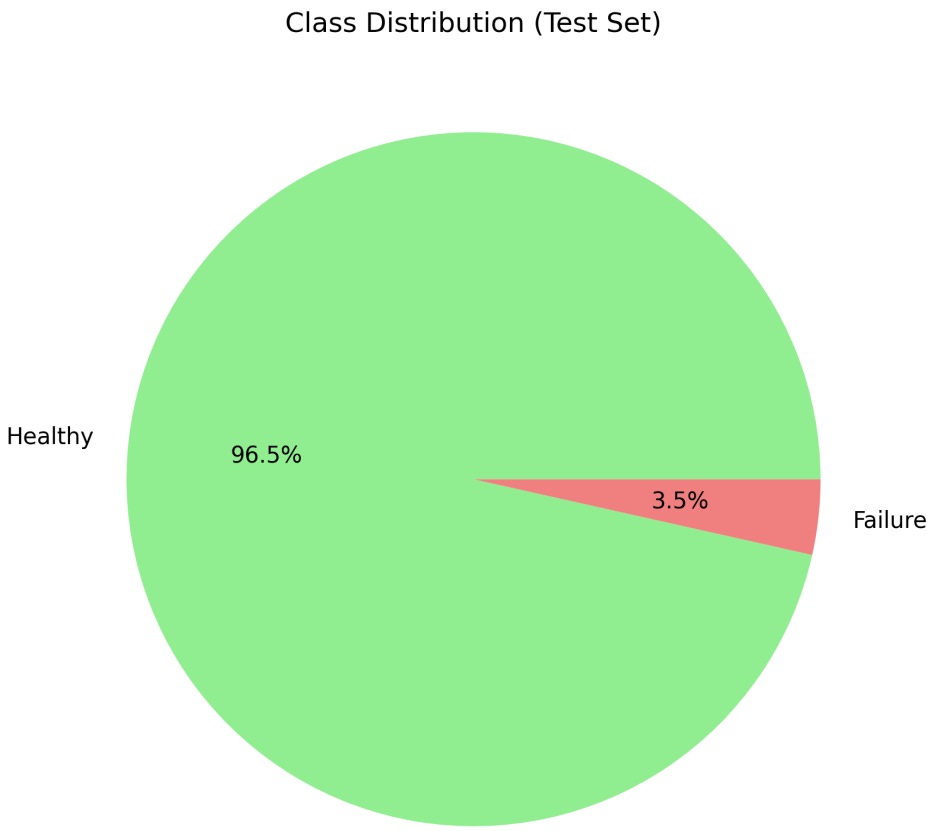

FIGURE 1.1 – Distribution des classes après nettoyage.

1.3 Solution proposée

La solution est organisée en quatre couches :

— un pipeline de données pour découvrir, nettoyer, transformer et exporter les informations ;

— une couche Machine Learning pour entraîner, comparer et sauvegarder les modèles ;

— une API FastAPI pour servir les prédictions et les données dashboard ;

— un frontend clair composé du dashboard, du ML Lab, de la présentation et de la page rapport.

L'utilisateur saisit les conditions de la machine. Le backend reconstruit les variables d'ingénierie, charge le modèle actif, calcule la probabilité de panne, applique le seuil 0.25 et retourne le niveau de risque ainsi que l'action recommandée.

Chapitre 2

Données, ETL et Nettoyage

2.1 Sources brutes

Deux fichiers bruts sont utilisés : `machine_predictive_maintenance.csv` et `ai4i_2020_predictive_maintenance.csv`. Chaque fichier contient 10 000 observations. Les deux sources décrivent la même population de machines, mais avec des colonnes et des labels différents.

Les variables principales sont : type de produit, température de l'air, température du processus, vitesse de rotation, couple, usure de l'outil et indicateurs de panne.

2.2 Fusion canonique

Les deux fichiers ne sont pas empilés, car cela créerait des doublons et un risque de fuite de données. Ils sont fusionnés sur des clés stables : UDI, product id, type de produit, températures, vitesse, couple et usure. Les colonnes sont ensuite renommées dans un schema clair : `air_temp_k`, `process_temp_k`, `rotational_speed_rpm`, `torque_nm`, `tool_wear_min`, `machine_failure`.

2.3 Résultats du nettoyage

Après nettoyage, le dataset contient 10 000 lignes et 17 colonnes. Les incohérences restantes sont nulles. La répartition des familles de pannes est la suivante :

Famille de panne	**Nombre de lignes**
No Failure	9 652
Heat Dissipation Failure	112
Power Failure	95
Overstrain Failure	78
Tool Wear Failure	45
Random Failures	18

Table 2.1 – Répartition des pannes après nettoyage.

2.4 Séparation train, validation et test

La séparation est stratifiée pour conserver le taux de panne dans chaque partition. Le train contient 6 000 lignes, la validation 2 000 lignes et le test 2 000 lignes. Les taux de panne restent proches de 3.5%.

Jeu	**Lignes**	**Taux de panne**
Train	6 000	3.483%
Validation	2 000	3.450%
Test	2 000	3.500%

Table 2.2 – Séparation stratifiée des données.

Chapitre 3

Feature Engineering

3.1 Objectif

Le feature engineering transforme les mesures brutes en variables plus informatives. Les variables construites restent interprétables pour un contexte industriel : stress thermique, charge mécanique, usure, couple élevé et faible vitesse.

3.2 Variables finales

Le modèle utilise 16 variables : 13 numériques et 3 catégorielles. Les principales variables construites sont :

Variable	**Formule / règle**	**Sens métier**
`temp_delta_k`	process temp - air temp	Écart thermique.
`power_proxy`	vitesse $\times$ couple	Charge mécanique approximée.
`torque_speed_ratio`	couple / vitesse	Stress à faible vitesse.
`thermal_load`	delta temp $\times$ couple	Charge thermique et mécanique.
`wear_temp_stress`	usure $\times$ delta temp	Outil usé sous chaleur.
`high_torque_flag`	couple ≥ 55	Indicateur de couple élevé.
`low_speed_flag`	vitesse ≤ 1400	Indicateur de faible vitesse.
`wear_risk_band`	usure catégorisée	fresh, moderate, elevated, critical.

Table 3.1 – Variables construites principales.

3.3 Analyse de corrélation

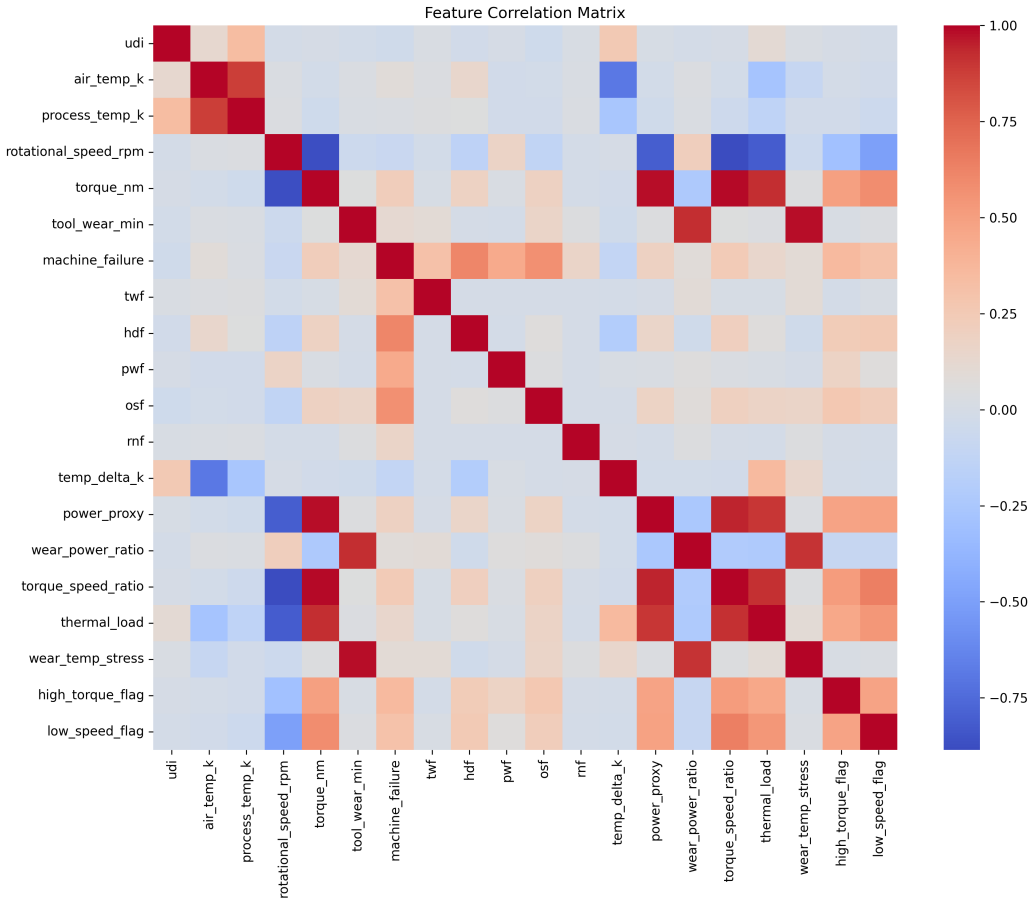

FIGURE 3.1 – Matrice de corrélation des variables.

La matrice montre les relations entre variables, notamment les interactions entre vitesse, couple et température. Ces interactions justifient l'utilisation de modèles d'arbres, capables de capturer des effets non linéaires.

Chapitre 4
Modélisation et Sélection

4.1 Modèles comparés

Le pipeline compare LogisticRegression, RandomForest, ExtraTrees, GradientBoosting, NeuralNetwork_MLP et DeepLearning_TF. Les modèles classiques utilisent un pipeline de preprocessing : imputation, standardisation des variables numériques et encodage one-hot des variables catégorielles.

4.2 Leaderboard

Modèle	**Seuil**	**Val. BA**	**Test BA**	**Précision**	**Recall**
GradientBoosting	0.25	89.49%	90.38%	81.43%	81.43%
RandomForest	0.40	88.25%	88.42%	90.00%	77.14%
ExtraTrees	0.35	84.34%	84.69%	80.33%	70.00%
DeepLearning_TF	0.30	82.22%	80.33%	74.14%	61.43%
NeuralNetwork_MLP	0.85	80.87%	79.72%	79.25%	60.00%
LogisticRegression	0.90	78.26%	80.90%	48.39%	64.29%

Table 4.1 – Comparaison des modèles candidats.

4.3 Choix du modèle actif

GradientBoosting est retenu car il offre le meilleur profil de validation. Il capture les relations non linéaires entre couple, vitesse, température et usure. Le Deep Learning est documenté dans le pipeline et le ML Lab, mais sur ce dataset tabulaire, le modèle d'arbres boostés reste plus adapté.

Chapitre 5
Évaluation Finale

5.1 Métriques principales

Le modèle actif GradientBoosting atteint les résultats suivants sur l'évaluation finale :

Métrique	**Valeur**
Accuracy	98.80%
Balanced accuracy	89.05%
Précision	85.94%
Recall / pannes capturées	78.57%
F1 score	82.09%
ROC AUC	99.05%
Average precision	91.28%
Taux d'alerte	3.20%
Part des fausses alertes	14.06%

Table 5.1 – Métriques finales du modèle actif.

5.2 Matrice de confusion

	Prédit normal	**Prédit panne**
Réel normal	1 921	9
Réel panne	15	55

Table 5.2 – Matrice de confusion finale.

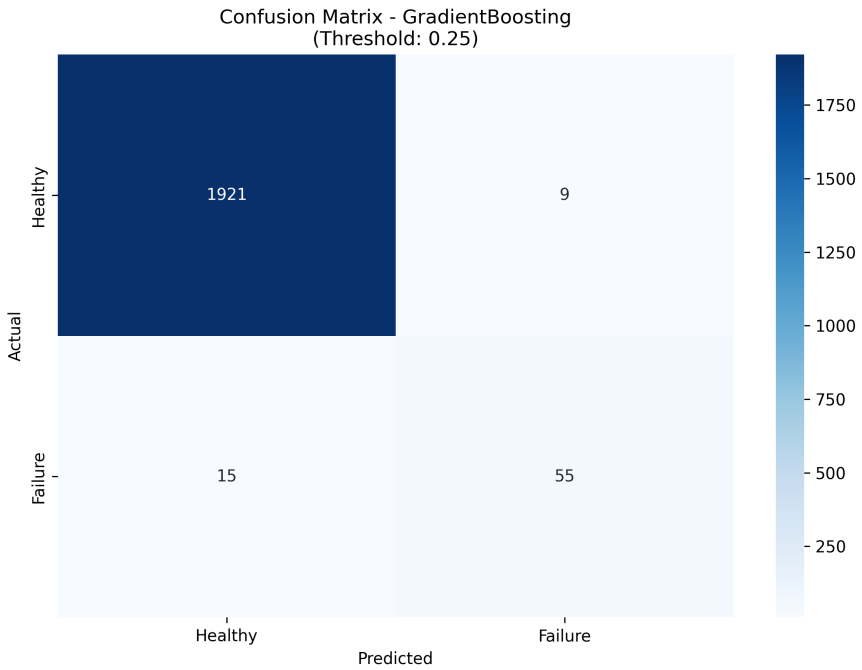

FIGURE 5.1 – Matrice de confusion générée.

Le modèle déclenche 64 alertes sur 2 000 cas de test. Parmi ces alertes, 55 correspondent à de vraies pannes et 9 sont des fausses alertes. Il manque 15 pannes. Cette lecture opérationnelle est plus utile que l'accuracy brute.

5.3 Courbe ROC et importance des variables

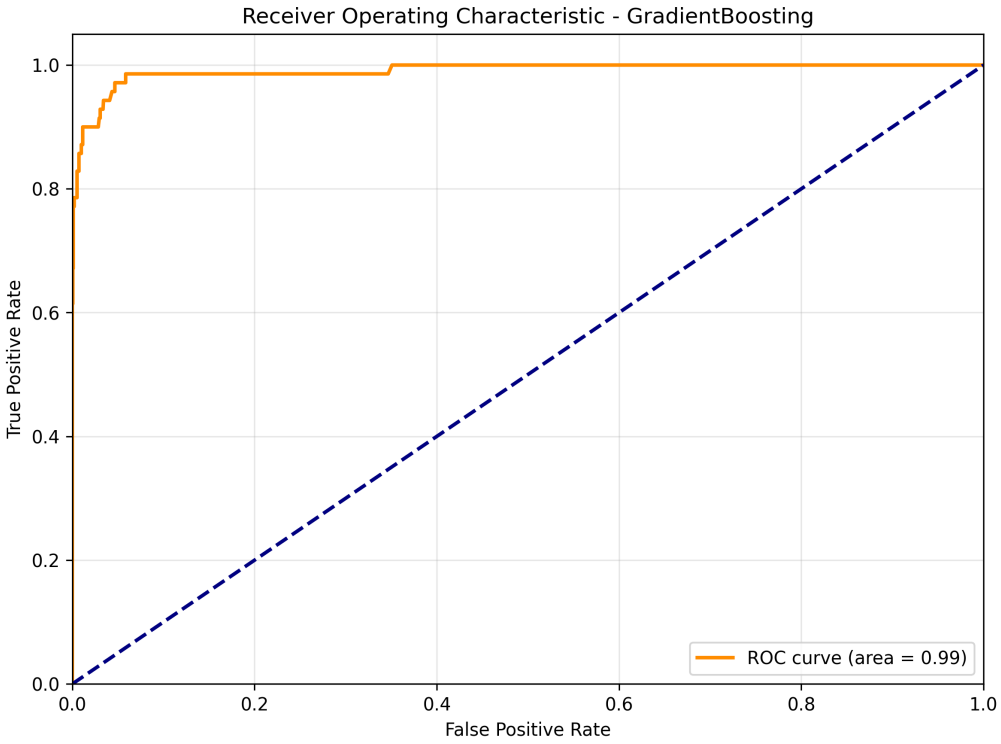

FIGURE 5.2 – Courbe ROC du modèle actif.

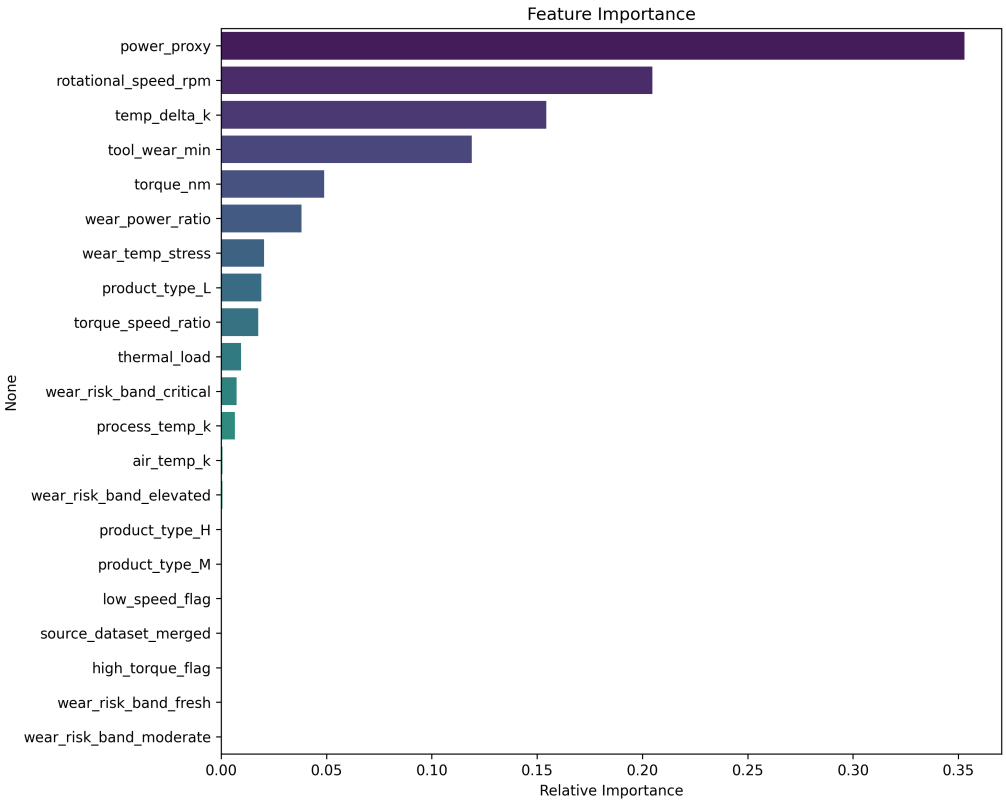

FIGURE 5.3 – Importance des variables.

La ROC AUC élevée montre que le modèle classe très bien les cas risqués. L'importance des variables permet d'expliquer les alertes aux utilisateurs.

Chapitre 6

API, Frontend et Synchronisation

6.1 API FastAPI

Le backend expose les routes principales : `/`, `/notebook`, `/presentation`, `/report`, `/model_summary`, `/dashboard`, `/api/models` et `/predict`. La route `/predict` reçoit les valeurs machine, reconstruit les variables d'ingénierie, charge le modèle actif et retourne la probabilité, le seuil, le modèle, la bande de risque et l'action recommandée.

6.2 Logique de prédiction

```
row["temp_delta_k"] = row["process_temp_k"] - row["air_temp_k"]
row["power_proxy"] = row["rotational_speed_rpm"] * row["torque_nm"]
row["thermal_load"] = row["temp_delta_k"] * row["torque_nm"]
row["high_torque_flag"] = int(row["torque_nm"] >= 55)
row["low_speed_flag"] = int(row["rotational_speed_rpm"] <= 1400)
```

Listing 6.1 – Reconstruction simplifiée des variables côté API.

6.3 Frontend

Le dashboard principal met le résultat en avant : probabilité de panne, niveau de risque, seuil, modèle et action. Le ML Lab conserve le contenu technique et les cellules exécutables, mais affiche les mêmes métriques que le dashboard. La présentation raconte le projet de manière courte. La page rapport intègre ce PDF.

6.4 Synchronisation

Toutes les vues utilisent les mêmes exports API : modèle actif GradientBoosting, seuil 0.25, balanced accuracy 89.05%, pannes capturées 78.57% et taux d'alerte 3.20%. Cette synchronisation évite les contradictions entre le dashboard, le ML Lab et la présentation.

Chapitre 7
Déploiement Railway

7.1 Préparation

Le projet est préparé pour Railway avec `railway.json` et `nixpacks.toml`. Railway utilise Python 3.11, installe les dépendances depuis `api/requirements.txt` et démarre le service avec Uvicorn.

```
uvicorn api.app:app --host 0.0.0.0 --port $PORT
```

Listing 7.1 – Commande de démarrage Railway.

La route `/health` sert de healthcheck. Le fichier `.gitignore` et le fichier `.railwayignore` excluent les fichiers inutiles au déploiement : compilateur LaTeX local, anciens PDF, caches Python et artifacts candidats volumineux.

7.2 Contenu déployé

Le déploiement conserve uniquement le nécessaire pour l'application : backend, frontend, exports JSON, rapport PDF, assets, données utiles au ML Lab et modèle actif Gradient-Boosting. Cela réduit la taille du projet et accélère l'upload Railway.

7.3 Commande CLI

Le déploiement s'effectue via Railway CLI avec un projet lié et un service web :

```
railway up --detach
```

Chapitre 8

Conclusion

Le projet fournit un workflow complet de maintenance prédictive : données brutes, nettoyage, feature engineering, entraînement, évaluation, API, frontend et rapport. Le résultat final est un outil compréhensible : l'utilisateur saisit les conditions d'une machine, obtient une probabilité de panne et reçoit une action claire.

Le choix de GradientBoosting est justifié par sa performance sur des données tabulaires déséquilibrées. La politique de seuil transforme le modèle en décision opérationnelle. Le frontend synchronisé rend le système plus cohérent et plus simple à utiliser. Le déploiement Railway rend l'application accessible comme service web.